



COMP 1039 – DESIGN PATTERNS WITH C++

Assignment 2 – Design Document

Adam Katic – A1706436

Complete Class Diagram & Overview

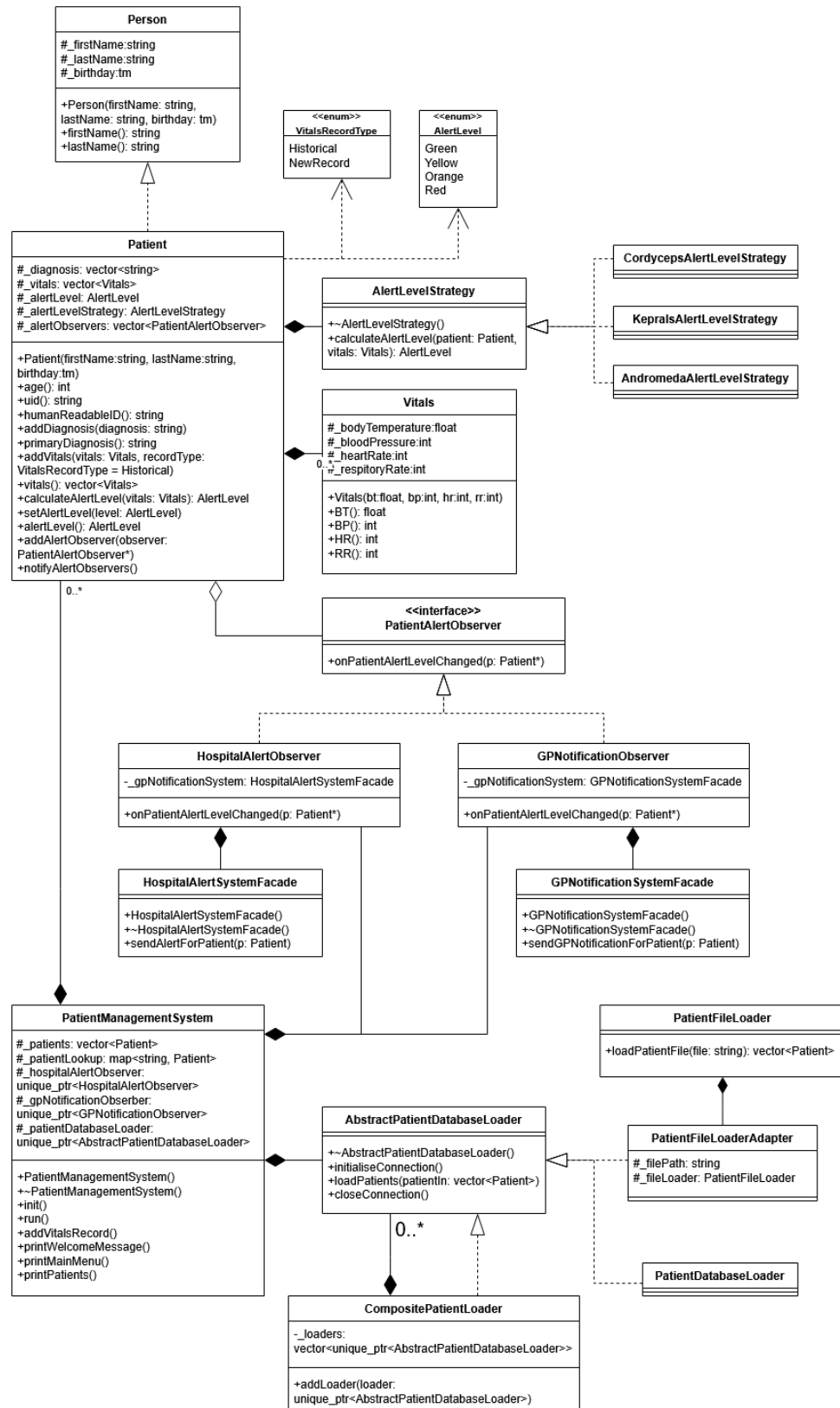


Figure 1 - Complete system class design after completing all functional requirements.

New classes/components created for this assignment:

- ***PatientFileLoaderAdapter***: An adapter to connect the *PatientFileLoader* to the abstract database loader. Adapts returned patients from the file loader to the interface expected by the *PatientManagementSystem*.
- ***CompositePatientLoader***: Handles multiple abstract data loaders, to allow the *PatientManagementSystem* to load patients from multiple loaders.
- ***AlertLevelStrategy***: An abstract interface that concrete strategies for setting alert levels are derived from.
- ***CordycepsAlertLevelStrategy***: Calculates the alert level for a patient with the Cordyceps Brain Infection, when new vitals are added.
- ***KepralsAlertLevelStrategy***: Calculates the alert level for a patient with Kepral's Syndrome, when new vitals are added.
- ***AndromedaAlertLevelStrategy***: Calculates the alert level for a patient with the Andromeda Strain, when new vitals are added.
- ***VitalsRecordType***: Enumeration that allows vitals records to be distinguished by the *PatientManagementSystem*, depending on if they were added from a database/file or added manually by the user.
- ***PatientAlertObserver***: Abstract observer that concrete hospital and GP observers are derived from.
- ***HospitalAlertObserver***: Observer registered with each patient, to monitor if that patient's alert level changes to *AlertLevel::Red* and alert the hospital if it does.
- ***GPNotificationObserver***: Observer registered with each patient, to monitor if that patient's alert level changes to *AlertLevel::Red* and alert their GP if it does.

Other pre-existing classes, including *Patient* and *PatientManagementSystem* were also modified during the assignment. Details about their changes are mentioned in each functional requirement section.

FR1: Load patients from file

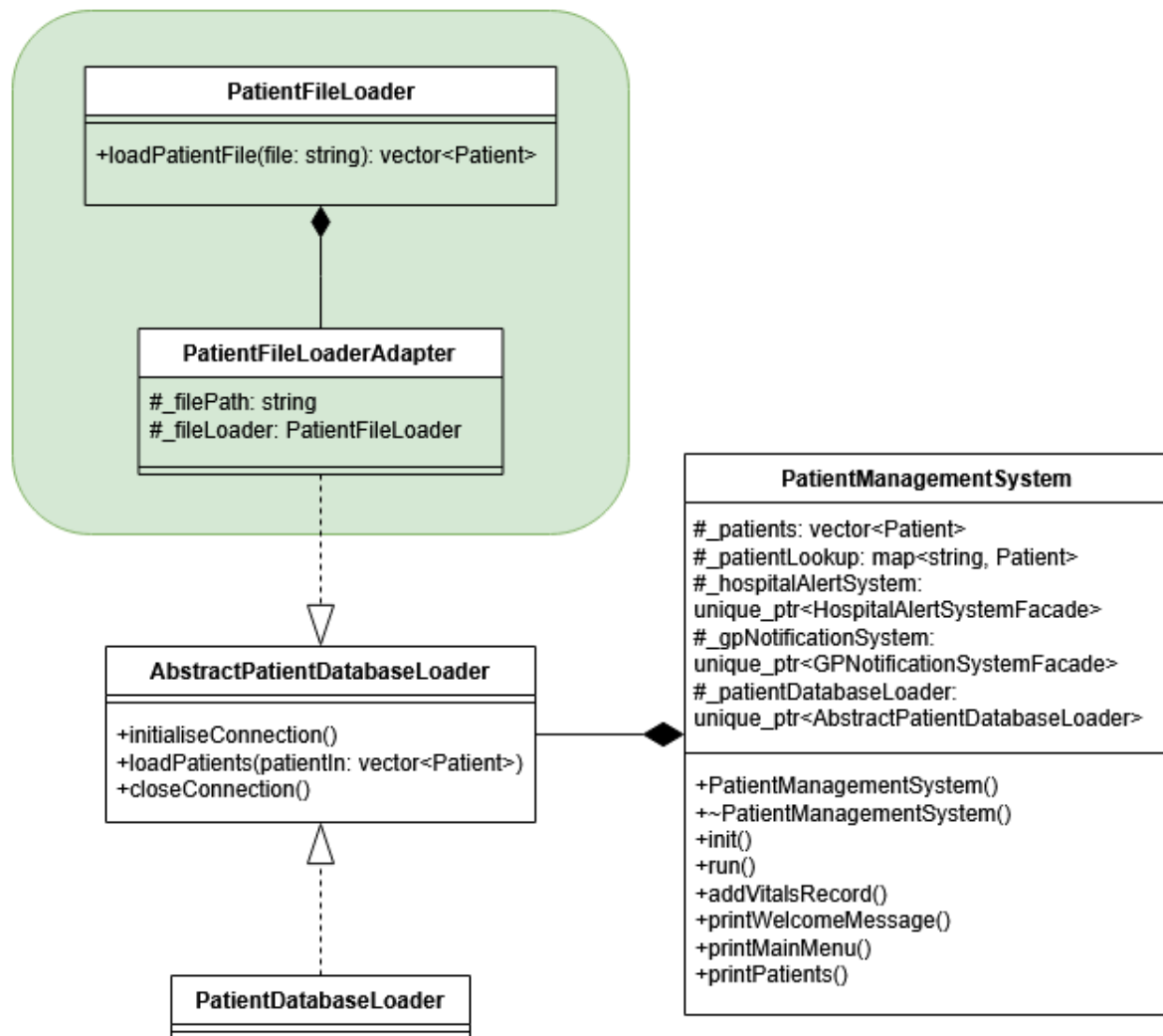


Figure 2 - Adapter design pattern for file loader implementation.

Chosen design pattern: Adapter design pattern

The system has to be able to load patient data from the given patients.txt file. However, the patient management system already utilises an existing database loader, so to prevent a rework of the management system I had to use the adapter pattern and create a file loader adapter that implements the existing abstract database loader similarly to the existing database loader. This then interacts with the provided patient file loader to parse the text file and create *Patient* objects, which are then forwarded onto the management system.

With this implementation, the patient management system only needs a single line of code changed to use the new file loader.

How it works:

1. After changing the single line of code in *PatientManagementSystem*, the loader stored in the unique pointer is now *PatientFileLoaderAdapter* instead of a *PatientDatabaseLoader*.

2. The *PatientManagementSystem* will initialise the *PatientFileLoaderAdapter* by calling *initialiseConnection()*, the same way as with the *PatientDatabaseLoader*.
3. The *PatientManagementSystem* then calls *loadPatients(...)* through the *PatientFileLoaderAdapter*, which is then delegated to the *PatientFileLoader*.
4. The *PatientFileLoader* then reads the given file path argument, loads the file and then for each line:
 - a. Splits the line into UID, name, birthday, diagnosis, and vitals strings.
 - b. Splits the name into first and last name, transforms the birthday into the correct type, then constructs a new patient from this data.
 - c. Adds the given diagnosis to the patient.
 - d. If vitals exist: split each record, convert to float/int, then add to the patient in a new Vitals object.
 - e. Push this into vector and read the next patient.
5. Once all patients have been read, the *PatientFileLoader* then returns the vector of patients back to the *PatientFileLoaderAdapter*, which are then appended to the vector passed by the *PatientManagementSystem*.
6. The *PatientManagementSystem* will then call *closeConnection()* for the file loader.

Git commits:

- Commits **5d430fb1**, **08c7042c**, and **cdd2997b** cover implementation of *PatientFileLoader*.
- Commits **65d31fd0**, **35361d35**, and **b0bed439** implement the header and source files for *PatientFileLoaderAdapter*.
- Commit **a31f886e** modifies *PatientManagementSystem* to use the new file loader.
- Commit **2e2aadcc** simply deleted a misspelled file, I probably could have amended the commit instead.
- Commit **ae8975f5** fixes an encoding issue I had, where curly apostrophes were not printing correctly when running the application.

FR2: Load Patients from file and database

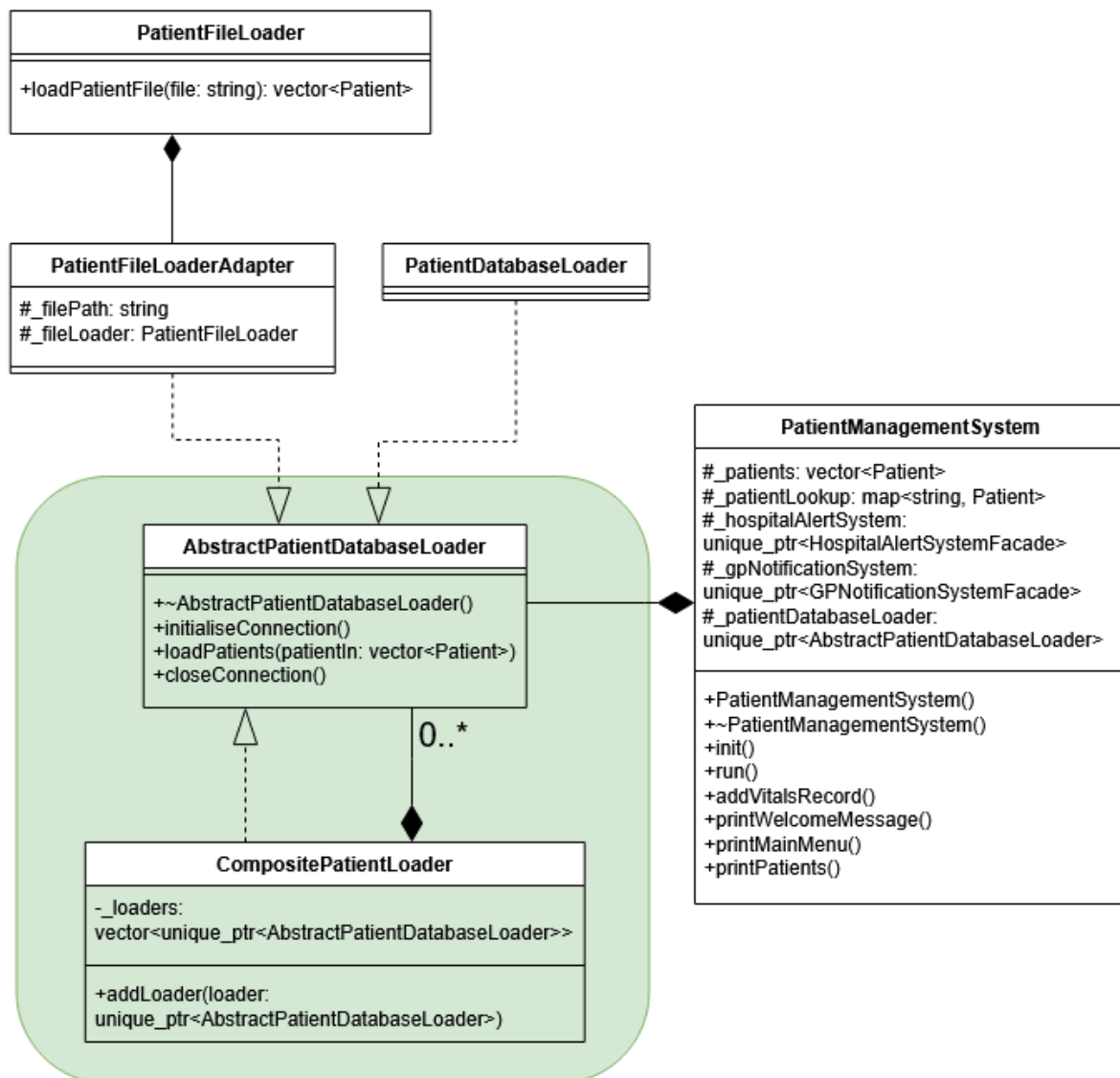


Figure 3 - Composite design pattern for database & file loading.

Chosen design pattern: Composite design pattern

The *PatientManagementSystem* still needs to load patients from the database in addition to loading them from a file. To do this, I applied the composite pattern by introducing a new *CompositePatientLoader* class, which allows *PatientManagementSystem* to treat a group of loaders as a single loader instead. Both the file and database loaders can be stored within the composite loader, which in turn calls each loader's methods. To make it easier to switch between file loading, database loading, and combined loading, I also added a helper function in *PatientManagementSystem* to create each loader and read patients in the correct order.

The *CompositePatientLoader* class owns its children, so a virtual destructor was added to *AbstractPatientDatabaseLoader* so that child loaders stored through the abstract loader are destroyed safely through the base-class interface.

How it works:

1. *PatientManagementSystem* creates and owns a *CompositePatientLoader* through the *AbstractPatientDatabaseLoader* interface.
2. A *PatientDatabaseLoader* and a *PatientFileLoaderAdapter* are then added to the *CompositePatientLoader*.
3. The *CompositePatientLoader* calls *initialiseConnection()*, *loadPatients(...)*, and *closeConnection()* on both child loaders, which calls the respective versions of those methods within each child.
4. *PatientDatabaseLoader* patients are loaded first, followed by patients from the *PatientFileLoader*.
5. All patients are then returned to the *PatientManagementSystem* which proceeds as normal.

Git commits:

- Commits **69ebf1f2**, **bf38c05e**, and **f73aea7d** construct the header and source files for the composite patient loader.
- Commit **911ac1e6** adds a helper function to the patient management system, so only one line of code needs to be changed to revert back to database/file loading.
- Commit **877373fd** adds a virtual destructor to the abstract database loader, to help with memory management.

FR3: Calculate the patient alert levels

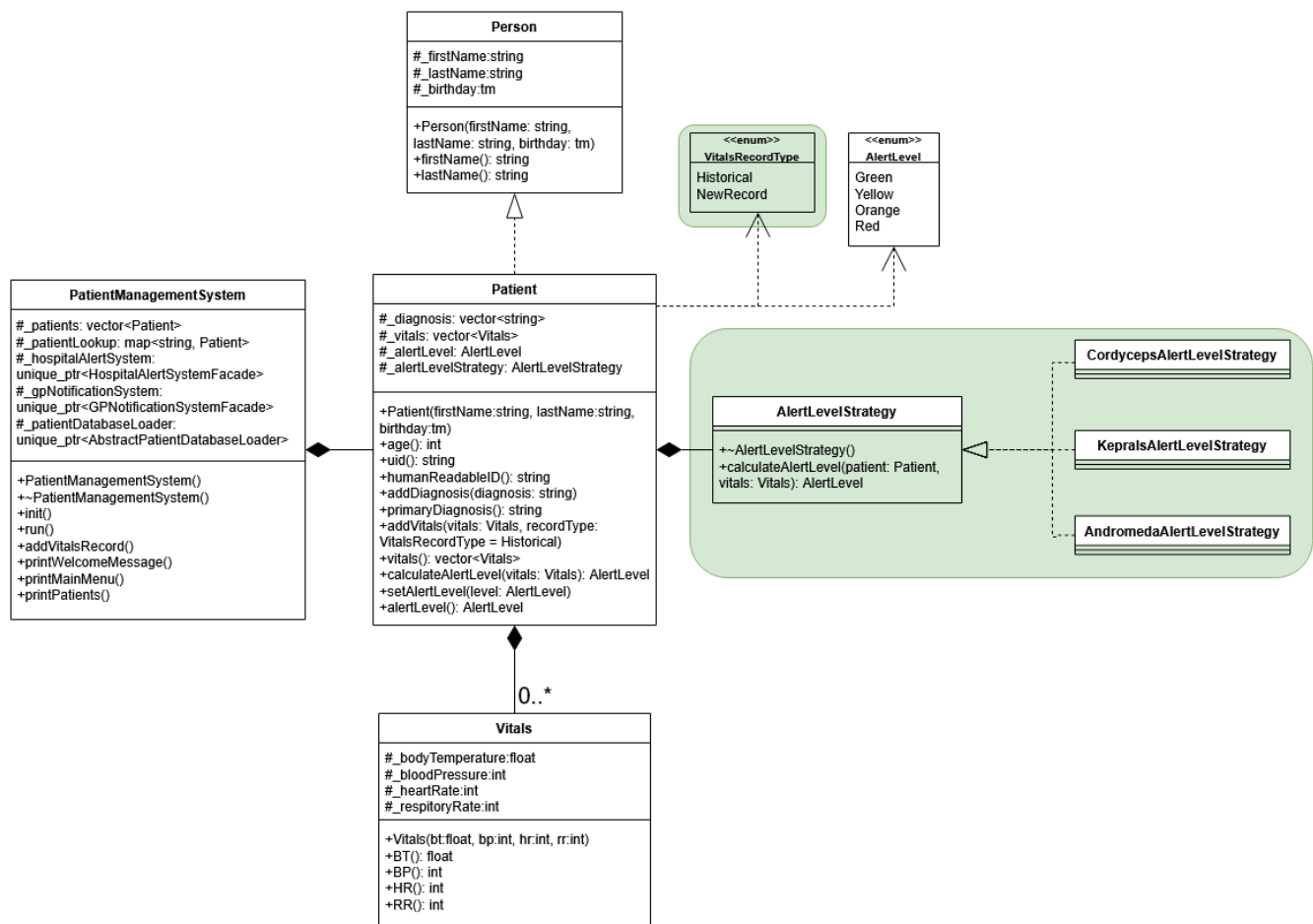


Figure 4 - Strategy design pattern for calculating patient alert levels.

Chosen design pattern: Strategy design pattern

The system needs to be able to calculate the alert level for patients, depending on their most recent vitals record. For this, I applied the strategy pattern, with a different strategy for each diagnosis, following the alert levels logic defined on the assignment page. The particular strategy is defined within *Patient*, depending on the received diagnosis. It is assumed for now that a patient can only have one diagnosis; if this wasn't the case, a vector of alert level strategies could be used, and the highest alert level overall used.

Instead of creating a new method in addition to *addVitals(...)*, I instead created a new enumeration to determine if a record is *Historical* or a *NewRecord*. This means that when a database or file is initially loaded, alert levels are not automatically calculated for them. Only vitals recorded at runtime calculate alert levels.

How it works:

1. When a database/file is loaded, *Patient* adds the listed diagnosis, along with the corresponding *AlertLevelStrategy*.
2. When vitals from a database/file are loaded, *addVitals(...)* is called with the default *VitalsRecordType::Historical* argument, skipping the alert level calculation.

3. When a user adds new vitals through the *PatientManagementSystem*, *VitalsRecordType::NewRecord* is passed as an argument, prompting the *Patient* to also call *calculateAlertLevel()*.
4. Depending on the patient's *AlertLevelStrategy*, the patient calls *calculateAlertLevel()* on either the Cordyceps, Kepral's Syndrome, or Andromeda Strain child strategy classes.
5. The child class then uses the defined logic to return either an *AlertLevel* of *Green*, *Yellow*, *Orange*, or *Red*.
6. The patient stores this alert level with *setAlertLevel(...)*, printing if that alert level is above *Green*.

Git Commits:

- Commits **3665057e** and **af8065a5** set up the strategy pattern, create all relevant header/source files, and implement the abstract interface.
- Commits **fc44bf22**, **b96d88d4**, **52228952**, and **a672fe50** override the *calculateAlertLevel()* method from the abstract interface, in each derived class. Logic for each derived class is also implemented.
- Commits **cfd5cd4f** and **775bab21** create a new enumeration to distinguish between historical and new records.
- Commits **6169af36**, **25bb137d**, and **d836ac31** modify the existing *Patient* and *PatientManagementSystem* to calculate/set alert based on the given diagnosis.

FR4: Alert the hospitals and GPs

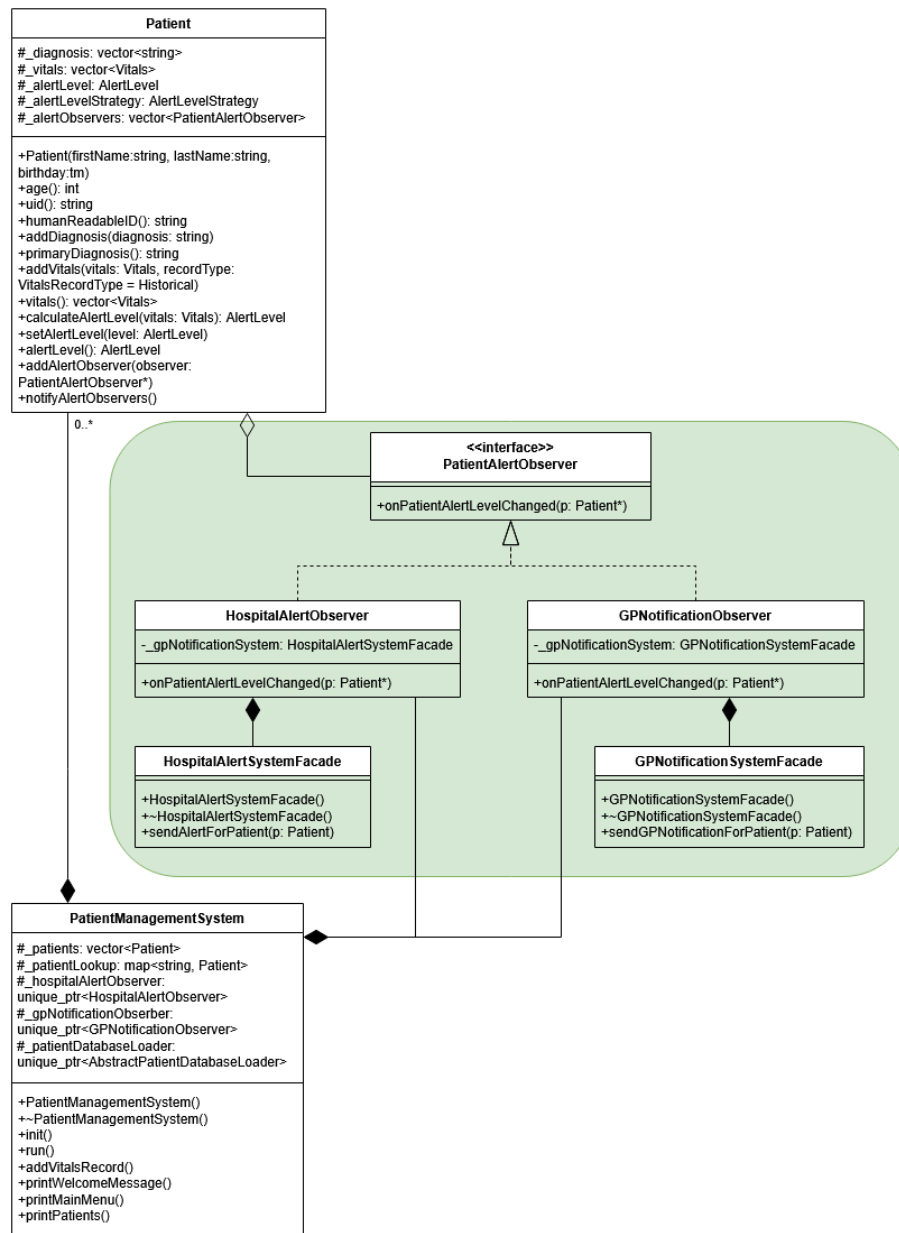


Figure 5 - Observer pattern for hospital and GP alerts.

Chosen design pattern: Observer design pattern

The *PatientManagementSystem* needs to communicate with both the hospital and the patient's GP whenever that patient's alert level changes to `AlertLevel::Red`. I used the observer pattern for this, as multiple observers can be registered for each patient through an abstract interface. When the patient's alert level goes to `AlertLevel::Red`, both the hospital and the patient's GP are notified with the same function call.

The initial *PatientManagementSystem* directly owned both the hospital alert and GP notification systems directly, however for this pattern I removed direct ownership of the facade objects and

added the observers in their place. *Patient* stores abstract pointers for the observers but does not outright own them, hence the association relationship. I also did not explicitly create a *removeObserver(...)* method for *Patient* as they are static and patients/observers are not removed during runtime.

How it works:

1. *PatientManagementSystem* creates *HospitalAlertObserver* and *GPNotificationObserver* objects.
2. After loading patients from the database/file, *PatientManagementSystem* then registers both observers with each patient through *addAlertObserver(...)*.
3. Whenever a new vitals record is added to a patient, that patient calculates their alert level. If the alert level is changed to *AlertLevel::Red*, *Patient* will then call *notifyAlertObservers()*.
4. Both *HospitalAlertObserver* and *GPNotificationObserver* will receive the event and call *onPatientAlertLevelChanged* to notify their respective facades.
5. *HospitalAlertSystemFacade* will call *sendAlertForPatient(...)*, while *GPNotificationSystemFacade* will call *sendGPNotificationForPatient(...)*.

Git Commits

- Commits **57578547** and **12e687af** create and implement the abstract and concrete observer classes.
- Commit **54102bfc** connects the old system facades to the new observers.
- Commits **29033a8f**, **50e756e8**, and **409666fb** modify the *Patient* class to store, create, and notify observers.
- Commits **34e743d4** and **769d3fac** update the *PatientManagementSystem* to own pointers to observer objects instead of the system facades.
- Commit **e7a2e594** modifies the *PatientManagementSystem* to add hospital and GP notification observers to all loaded patients.